Git och GitHub

Kommandolathund

Referensmaterial för introduktionskursen i Git och GitHub.
Använd lathunden när du behöver komma ihåg ett kommando eller förstå vad det gör.

> Grundprincip: Kontrollera med `git status` före och efter viktiga kommandon. Då ser du vilken branch du befinner dig på och vilka förändringar Git känner till.

Konfigurera Git

Kommando	Förklaring
`git config --global user.name "Your Name"`	Anger namnet som kopplas till dina commits.
`git config --global user.email "your.email@example.com"`	Anger e-postadressen som kopplas till dina commits.
`git config --global init.defaultBranch main`	Använder `main` som huvudbranch i nya repositories.
`git config --global --list`	Visar den globala Git-konfigurationen.

Skapa och hämta repositories

Kommando	Förklaring
`git init`	Skapar ett nytt lokalt Git-repository i den aktuella mappen.
`git clone <repository-url>`	Hämtar ett befintligt repository, exempelvis från GitHub, till datorn.

Kontrollera status och historik

Kommando	Förklaring
`git status`	Visar aktuell branch och vilka filer som har skapats, ändrats, tagits bort eller förberetts.
`git log`	Visar projektets commit-historik.
`git log --oneline`	Visar commit-historiken i ett kompakt format.
`git diff`	Visar ändringar som ännu inte ligger i staging area.

Kommando	Förklaring
`git diff --staged`	Visar de ändringar som kommer att ingå i nästa commit.

Förbereda förändringar

Kommando	Förklaring
`git add <filename>`	Lägger till en viss fil i staging area inför nästa commit.
`git add .`	Lägger till alla nya, ändrade och borttagna filer i den aktuella mappen.
`git rm <filename>`	Tar bort en fil och förbereder borttagningen inför nästa commit.
`git mv <old-name> <new-name>`	Byter namn på eller flyttar en fil och förbereder förändringen.

Skapa commits

Kommando	Förklaring
`git commit -m "Add login form"`	Skapar en commit av förändringarna i staging area.
`git commit --amend`	Ändrar den senaste commiten, exempelvis med ett nytt meddelande eller en glömd fil. Använd försiktigt om commiten redan har pushats.

Arbeta med GitHub

Kommando	Förklaring
`git remote -v`	Visar vilka fjärranslutna repositories projektet är kopplat till.
`git remote add origin <repository-url>`	Kopplar det lokala projektet till ett repository på GitHub.
`git push -u origin main`	Skickar `main` till GitHub och skapar en spårningskoppling. `-u` behövs normalt bara första gången.
`git push`	Skickar lokala commits till den kopplade branchen på GitHub.
`git pull`	Hämtar förändringar från GitHub och sammanfogar dem med aktuell branch.
`git fetch`	Hämtar information och commits från GitHub utan att sammanfoga dem.

Arbeta med branches

Kommando	Förklaring
`git branch`	Visar lokala branches. Den aktiva branchen markeras med en asterisk.
`git branch --all`	Visar både lokala och fjärranslutna branches.
`git switch -c feature/add-login`	Skapar en ny branch och byter direkt till den.
`git switch main`	Byter till branchen `main`.
`git branch feature/add-login`	Skapar en branch utan att byta till den.
`git branch -m <new-name>`	Byter namn på den aktuella branchen.
`git push -u origin feature/add-login`	Skickar en ny branch till GitHub och skapar en spårningskoppling.

Sammanfoga branches

Kommando	Förklaring
`git switch main`	Byt först till den branch som ska ta emot förändringarna.
`git merge feature/add-login`	För in förändringarna från `feature/add-login` i den branch du står i.
`git merge --abort`	Avbryter en pågående merge och återgår till läget före sammanslagningen.

Ta bort branches

Kommando	Förklaring
`git branch -d feature/add-login`	Tar bort en lokal branch som redan har sammanfogats.
`git branch -D feature/add-login`	Tvingar bort en lokal branch även om den inte har sammanfogats. Arbete kan gå förlorat.
`git push origin --delete feature/add-login`	Tar bort en branch från GitHub.

Ångra förändringar

Kommando	Förklaring
`git restore <filename>`	Återställer en fil till innehållet i senaste commiten. Osparade ändringar i filen försvinner.
`git restore --staged <filename>`	Tar bort en fil från staging area men behåller ändringarna i filen.
`git revert <commit-id>`	Skapar en ny commit som upphäver en tidigare commit. Passar bra för historik som redan har pushats.

Kommando	Förklaring
`git reset --soft HEAD~1`	Tar bort senaste commiten men behåller förändringarna i staging area.
`git reset HEAD~1`	Tar bort senaste commiten och behåller ändringarna i filerna, utanför staging area.
`git reset --hard HEAD~1`	Tar bort senaste commiten och dess förändringar. Använd mycket försiktigt.

Tillfälligt lägga undan förändringar

Kommando	Förklaring
`git stash`	Lägger tillfälligt undan ändringar som ännu inte har sparats i en commit.
`git stash list`	Visar sparade stashar.
`git stash pop`	Återställer den senaste stashen och tar bort den från stash-listan.

Vanliga arbetsflöden

Spara förändringar lokalt

```
git status
git add .
git commit -m "Add todo functionality"
```

Skicka commits till GitHub

```
git push
```

Hämta andras förändringar

```
git pull
```

Arbeta med en ny funktion

```
git switch main
git pull
git switch -c feature/add-todo
git add .
git commit -m "Add todo creation"
git push -u origin feature/add-todo
```

Viktiga begrepp

Kommando	Förklaring
`Repository`	Ett projekt och dess historik som hanteras med Git.
`Working directory`	Projektets filer som du arbetar med.
`Staging area`	Förberedda förändringar inför nästa commit.
`Commit`	En sparad kontrollpunkt i projektets historik.
`Branch`	En separat utvecklingslinje.
`main`	Projektets huvudsakliga branch.
`Remote`	Ett fjärranslutet repository, exempelvis på GitHub.
`origin`	Standardnamnet på projektets remote.
`Push`	Skickar lokala commits till GitHub.
`Pull`	Hämtar och sammanfogar förändringar från GitHub.
`Fetch`	Hämtar förändringar utan att sammanfoga dem.
`Merge`	Sammanfogar förändringar från olika branches.
`Clone`	Hämtar en lokal kopia av ett repository.
`Pull request`	Ett förslag om att granska och sammanfoga en branch.